BÖLÜM 2: PLANLAR VE SINIRLAR — KARMAŞIKLIĞI PARÇALARA AYIRMAK ⏱ 8 dk • DERS 7

Parça 7: Bağlaşıklık ve Uyum (Coupling vs. Cohesion): Sistemleri Yaşatan veya Öldüren İkiz Güç

Ön yüzde bir butonun rengini değiştirmenin fiziksel depodaki paketleme robotlarını durdurmaması için.

🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**

Büyük bir perakende uygulamasında kampanya döneminde bir kriz patlak verdi. Pazarlama ekibi açılış sayfasındaki tanıtım kutusunun metnini 'Ücretsiz Kargo' yerine 'Kargo Bedava' olarak güncelledi. Kod canlıya alındıktan on dakika sonra, fiziksel depodaki barkod okuyucu robotik vinçler durdu! Neden mi? Depodaki barkod sistemi, sepetin JSON çıktısını ayrıştırıyor ve kargo ağırlık kategorisini belirlemek için arayüzdeki banner metnini `*Kargo*` regex'iyle kontrol ediyordu! Metin değişince sistem tanımsız hata fırlattı ve tüm lojistik operasyonu kilitlendi. Bir pazarlama metni değişikliği koca fabrikayı durdurmuştu.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Bağlaşıklık (Coupling), modüller arasındaki bağımlılık derecesidir; Uyum (Cohesion) ise bir modülün içindeki fonksiyonların ne kadar tek bir amaca odaklandığıdır. Larry Constantine'in 1974'te formüle ettiği kural hala değişmemiştir: **Yüksek Uyum, Gevşek Bağlaşıklık (High Cohesion, Loose Coupling).** Birlikte değişmesi gereken şeyler aynı yerde yaşamıyorsa düşük uyum vardır. Modül A'daki tek bir satırı değiştirmek Modül B'yi bozuyor veya yeniden dağıtılmasını gerektiriyorsa sıkı bağlaşıklık vardır.

Temel Mimari İlkeler ve Zihinsel Modeller

Bağlaşıklık Türleri (Masumdan Zehirliye)

- **Veri Bağlaşıklığı (Sağlıklı):** Modüller birbirine sadece işi yapacak temel parametreleri geçirir.
- **Kontrol Bağlaşıklığı (Şüpheli):** Modül A, Modül B'ye dahili yürütme akışını dikte eden bayraklar (`ozelLoglamaYap=true`) gönderir.
- **Ortak/Veritabanı Bağlaşıklığı (Tehlikeli):** Birden fazla modül aynı paylaşılan veritabanı tablolarını doğrudan okur ve yazar.
- **İçerik Bağlaşıklığı (Ölümcül):** Modül A, Modül B'nin özel (private) değişkenlerine erişip içeriden müdahale eder.

Demeter Yasası (En Az Bilgi İlkesi)

Yabancılarla konuşmayın: `siparis.getMusteri().getAdres().getPostaKodu().vergiHesapla()` çağrısı çağıranı 4 seviye derinliğindeki iç yapıya sıkı sıkıya bağlar. Bunun yerine `siparis.vergiHesapla()` çağrılmalı ve iç gezintiyi nesne kendisi yönetmelidir.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Saçma Sapan Ameliyat (Shotgun Surgery):** Tek bir iş kuralını değiştirmek için 5 farklı serviste 18 ayrı dosyayı düzenlemek zorunda kalmak.
- **Ortak Çöp Çekmecesi (Common/Utils):** Şirketteki her servisin import ettiği, içinde 40.000 satır ilgisiz kod barındıran kontrolsüz `CommonUtils` kütüphaneleri.

- **Gezgin Veri (Tramp Data):** Sırf en alttaki fonksiyon bir `kullanici_id`ye ihtiyaç duyuyor diye 12 katman boyunca 50 alanlık koca bir bağlam nesnesini elden ele gezdirmek.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Git geçmişinde değişiklik sıklığı (churn) analizi yap: Neredeyse her pull request'te düzenlenen ilk 5 dosyayı bul. Orası senin toksik bağlaşıklık merkezindir.
> - **2. Adım:** En çok sürtüşme yaratan noktayı bir arayüz (interface) veya olay (event) arkasına alarak soyutla.
> - **3. Adım:** Ortak `Common` paketini dağıt; o kodları gerçek sahiplerinin modüllerine taşı.

Mimari Çıkarımlar ve Hile Tablosu

- Ekip büyüdükçe yazılım geliştirme hızının yavaşlamasının 1 numaralı sebebi sıkı bağlaşıklıktır.
- Uyum odaklanmakla ilgilidir: Bir sınıf veya servis tek bir işi yapmalı ve onu eksiksiz yapmalıdır.
- İki servisin her zaman birlikte canlıya alınması gerekiyorsa, onlar iki servis değil; arasına ağ kablosu çekilmiş tek bir servistir.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Git reponuzun son 6 aylık geçmişini tarayın. En sık değiştirilen dosya hangisi? Neden bu kadar çok farklı özellik o dosyaya dokunmak zorunda kalıyor?